

MEDIATEK

MT8676 Yocto UART

Version: 1.1
Release date: 2025-03-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Liliang Chen	Official release
1.1	2025-03-12	Liliang Chen	Modified Section 1.3.4 Add GPIO Setting

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 UART 4

1.1 Overview 4

1.1.1 Introduction 4

1.1.2 Abbreviation 4

1.2 Architecture/Process Overview 4

1.2.1 UART Introduction 4

1.2.2 MT8676 UART Feature 5

1.3 Configuration/Customization Guideline 6

1.3.1 Linux Build Configuration 6

1.3.2 Add UART DTS Node 6

1.3.3 Add APDMA DTS Node 6

1.3.4 Add GPIO Setting 7

1.3.5 Testing and Debugging 8

1.4 Frequently Asked Questions/Troubleshooting 8

1.4.1 UART Unable to Input/Output 8

1.4.2 UART Garbled Code 8

1.4.3 UART Does Not Print Kernel Logs 9

Exhibit 1 Terms and Conditions 10

List of Figures

Figure 1-1. Pin connections between SoC UART and device UART 4

List of Tables

Table 1-1. Abbreviations 4

1 UART

1.1 Overview

1.1.1 Introduction

This chapter introduces the hardware, software, and functions of the MT8676 UART controller.

1.1.2 Abbreviation

Table 1-1. Abbreviations

Abbreviation	Explanation
CTS	Clear To Send
DMA	Direct Memory Access
FIFO	First In, First Out
RTS	Request To Send
RX	Receiver
TX	Transmitter
UART	Universal Asynchronous Receiver/Transmitter

1.2 Architecture/Process Overview

1.2.1 UART Introduction

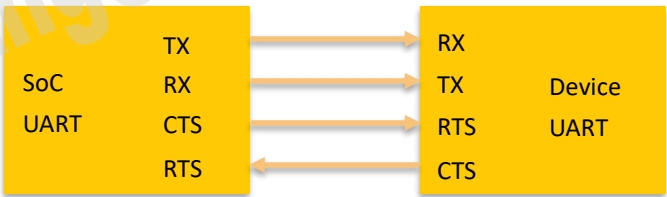


Figure 1-1. Pin connections between SoC UART and device UART

UART is a serial communication interface protocol commonly used for computers or microcontrollers to communicate with peripheral devices. TX, RX, CTS and RTS are four important signals in UART protocol, which are used to transmit/receive data and control the transmission process respectively.

- **TX (Transmit)**

TX is the transmit port in UART, which is the port used by computers or microcontrollers to send data. TX converts the data to be sent into serial data frames bit by bit, and sends it out on the communication line through the serialized data level. When the transmission is completed, the TX state will turn to low level.

- **RX (Receive)**

RX is the receiving port in UART, which is the port used by computers or microcontrollers to receive data. RX deserializes the data sent by the sender and converts it into a data frame to be received. When the data is received, the RX state will turn high.

- **CTS (Clear To Send)**

CTS is an output port that transmits a status message to the sender to indicate whether the RX end is ready to receive new data. When and only when the CTS state is high, it means that the RX end is ready to receive data; when the CTS state is low, it means that the RX end is processing data, and the sender should suspend transmission.

- **RTS (Ready To Send)**

RTS is an input port that transmits a status message to the receiver to indicate whether the TX end is ready to send new data. When the TX end is ready to send data, the RTS state is low; when the TX end is not ready to send data, the RTS state is high, and the receiver should suspend receiving.

The role of CTS and RTS is to perform handshake negotiation before data transmission to avoid the sender and receiver competing for the same time to transmit/receiving data, thereby ensuring the reliability and stability of data transmission.

In summary, the four signals of UART, TX, RX, CTS and RTS, are the core signals in UART protocol. Their combined use can realize serial transmission of data and effective flow control management.

Figure 1-1 shows an example of hardware connection between SoC and peripheral devices for data transmission via UART, where RTS and CTS are used for hardware flow control management and can be removed or retained based on the usage scenario.

1.2.2 MT8676 UART Feature

- Provides four serial ports
- UART0 /UART1 are 2-pin (TX, RX) UART channels.
- UART2/UART3 is a 4-pin (TX, RX, CTS, RTS) UART channel.
- UART3 mounts a UARTHUB, and the default mode is UARTHUB.
- Supports M16C450 and M16550A operation modes
- Compatible with standard software drivers
- Transmission system: Asynchronous
- Data length: 5 to 8 bits
- Hardware flow control: Automatic sending and receiving control based on CTS/RTS
- Software flow control: Use special characters Xon/Xoff for software flow control
- Baud rate programmable from 300 bps to 3 Mbps
- Interrupt request: Receive interrupt/send interrupt
- Data transfer: Support DMA (send/receive) transfer

1.3 Configuration/Customization Guideline

1.3.1 Linux Build Configuration

Configuration items:

- Assign MAX UART port numbers supported under Linux Kernel

```
CONFIG_SERIAL_8250_NR_UARTS=4
CONFIG_SERIAL_8250_RUNTIME_UARTS=4
```

- Enable UART DMA support

```
CONFIG_DMA_MTK_UART=y
```

- Enable 8250 UART protocol support and MTK UART driver

```
CONFIG_SERIAL_8250=y
```

1.3.2 Add UART DTS Node

Device node of UART:

```
uart0: serial@11001000 {
    compatible = "mediatek,mt6577-uart";
    reg = <0 0x11001000 0 0x1000>;
    interrupts = <GIC_SPI 250 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&clk26m>, <&infracfg CLK_INFRA_UART0>;
    clock-names = "baud", "bus";
    dmas = <&apdma 0 &apdma 1>;
    dma-names = "tx", "rx";
};
```

1.3.3 Add APDMA DTS Node

Device node of APDMA:

```
apdma: dma-controller@11300b80 {
    compatible = "mediatek,mt6985-uart-dma";
    reg = <0 0x11300b80 0 0x80>,
        <0 0x11300c00 0 0x80>,
        <0 0x11300c80 0 0x80>,
        <0 0x11300d00 0 0x80>,
        <0 0x11300d80 0 0x80>,
        <0 0x11300e00 0 0x80>,
        <0 0x11300e80 0 0x80>,
        <0 0x11300f00 0 0x80>;
    interrupts = <GIC_SPI 219 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 220 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 221 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 222 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 223 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 224 IRQ_TYPE_LEVEL_HIGH 0>;
};
```

```

                                <GIC_SPI 225 IRQ_TYPE_LEVEL_HIGH 0>,
                                <GIC_SPI 226 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&pericfg_ao_clk CLK_PERAOP_DMA_BCLK>;
    clock-names = "apdma";
    dma-requests = <8>;
    #dma-cells = <1>;
};

```

1.3.4 Add GPIO Setting

- Configure GPIO and set it to on by default:

```

&uart0 {
    pinctrl-names = "default", "sleep ";
    pinctrl-0 = <&uart0_pin_default>;
    pinctrl-1 = <&uart0_pin_sleep>;
    status = "okay";
};

```

- If not used in the OS, please set the status to off:

```

&uart0 {
    pinctrl-names = "default", "sleep ";
    pinctrl-0 = <&uart0_pin_default>;
    pinctrl-1 = <&uart0_pin_sleep>;
    status = "disabled";
};

```

- Configure default/sleep pins related to UART communication in PIO nodes:

```

&pio {
    uart0_pin_default: uart0_pin_uart_mode {
        pins_rx {
            pinmux = <PINMUX_GPIOxxx__FUNC_URXD0>;
            input-enable;
            bias-pull-up;
        };
        pins_tx {
            pinmux = <PINMUX_GPIOxxx__FUNC_UTXD0>;
            output-high;
        };
    };

    uart0_pin_sleep: uart0_pin_gpio_mode {
        pins_rx {
            pinmux = <PINMUX_GPIOxxx__FUNC_GPIOxxx>;
            bias-pull-down;
        };
        pins_tx {
            pinmux = <PINMUX_GPIOxxx__FUNC_GPIOxxx>;
            bias-pull-down;
        };
    };
};

```

}

1.3.5 Testing and Debugging

1. Set the baud rate to 921600, 8 data bits, 1 stop bit, no parity, like UART1

```
stty -F /dev/ttyS1 ispeed 921600 ospeed 921600 cs8
```

2. Get the current UART baud rate and data bit configuration information

```
stty -F /dev/ttyS1 -a
```

3. Use the following command to receive UART1 data and print it out through the console

```
cat /dev/ttyS1
```

4. Use the following command to let UART1 send a string "123"

```
echo 123 > /dev/ttyS1
```

1.4 Frequently Asked Questions/Troubleshooting

1.4.1 UART Unable to Input/Output

1. Please check whether the dts configuration is correct according to the information provided earlier.
2. Please use the Linux command to check whether the UART pin mode is switched to UART mode.
 - Check the status of PIN in Kernel, enter the following command:

```
# cat /proc/mtk_gpio/soc.pinctrl
# cd /proc/mtk_gpio/
# cat soc.pinctrl
```

e.g.,

```
# cat /proc/mtk_gpio/soc.pinctrl
PIN: [MODE] [DIR] [DOUT] [DIN] [PULL_EN] [PULL_SEL] [IES] [SMT] [DRIVE] ( [R1] [R0] )
0: 0 0 0 0 1 0 1 0 0
1: 0 0 0 0 1 0 1 0 0
2: 0 1 1 1 1 0 1 0 0
3: 6 0 0 0 1 0 1 0 0
```

1.4.2 UART Garbled Code

1. Please check whether the baud rate is consistent according to the information provided above.
2. Please check if the UART source_clk is as expected, usually source_clk is set to 26 MHz.

1.4.3 UART Does Not Print Kernel Logs

By default, the user version image will no longer print the Kernel log after entering the shell, while the engine/user-debug image will continue to print the Kernel log by default. You can refer to the following method to modify it.

- echo 1 > /proc/mtprintk: only can enable UART log after adb shell can work
- setprop persist.uartconsole.enable 1: can enable UART log after boot
- Enable UART logging by default for users by modifying codes

```
vendor/mediatek/proprietary/bootable/bootloader/lk/app/mt_boot/mt_boot.c  
cmdline_append("printk.disable_uart=0"); // BUILD_TYPE_USER
```

```
device/mediatek/mt6873/init.mt6873.rc mask "write /proc/bootprof 0"  
# write /proc/bootprof 0
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.